

WEEK 12 · TYPESCRIPT & MOCK SURGERY 2

THE COMPILER READS IT TOO

The application is finished. Today a compiler — TypeScript's, called `tsc` — reads one layer of it, and then you work alone for forty minutes.

WHERE WE ARE

Week 12 of 12 (week 13 is the exam). Six files, one loop, live data — and one layer where a value you did not write comes in.

THE NEXT 180 MINUTES

Four cycles, you typing nearly every piece (the long blocks are marked **copy this**): rename and guard · type the DOM · Claude migrates a module · one refactor in three commits. Break at 85, the surgery at 115, the assignment at 165.

WHAT THIS UNLOCKS

The project's last requirement, submitted this week — and the third verb on the exam paper.

Types describe shape, not intent.

TARGET · MINUTE 0

THIS IS WHERE YOU WILL BE AT MINUTE 180

the pantry, with its network layer in TypeScript – it looks exactly like the one you start from

reloadopen

המזווההרשימהפריטסיכום

המזווה

שםכמותמדףהוסף

חיפושמדףמיוןהכוללפי שם

מוצג: 0

נקה הכול

המזווההרשימהפריטסיכום

המזווה

שםכמותמדףהוסף

חיפושמדףמיוןהכוללפי שם

מוצג: 0

נקה הכול

המזווההרשימהפריטסיכום

המזווה

שם

375 · the same shape, narrower

► **THE POINT** — the screen does not change today. What changes is that `tsc` reads `ts/api.ts`, and a catalogue row with no title can no longer reach the list unchecked. You type each piece once in a cycle first, on the class lending library.

AI THIS WEEK

PARTNER. IT MIGRATES; YOU READ WHAT IT SILENCED.

LAYER · WEEKS 8–12 · PARTNER

ALLOWED

Claude writes the migration — in Claude Code on the folder, or in the chat with the files pasted — **against four rules you wrote into the prompt**, and you check what came back with `grep` and `tsc` before you run it (cycle 3; cycle 2's you-do).

NOT ALLOWED

`any`, `as` on a value from outside, `@ts-ignore` — three ways to switch the compiler off, each explained today — in anything you submit. **Any AI in cycles 1 and 4's "you do" and in the surgery** — alone, offline, like week 13.

NO SUBSCRIPTION?

Everything works in the free chat, and nothing in the course depends on a paid plan. Paste the files, copy back what returns, run the same review. Same grade.

A TYPE IS A CLAIM. IT IS NOT CODE.

TypeScript = JavaScript + type notes. `tsc` checks, then **erases** them.

the same module, before and after `tsc`

reloadopen

THE SAME MODULE, BEFORE AND AFTER `tsc`

highlight what disappearedclear

ts/source.ts — WHAT YOU WRITE

```
// Everything in this file that describes a type is gone after compilation.  
// (The class lending library's types – cycle 1 types a longer version of this file.)  
export type LookupKind = 'offline' | 'status' | 'shape';  
  
export interface ShelfEntry {  
  readonly id: string;  
  title: string;  
  added: string;  
}  
  
function isRecord(value: unknown): value is Record<string, unknown> {
```

js/source.js — WHAT THE BROWSER LOADS

```
function isRecord(value) {  
  return typeof value === 'object' && value !== null && !Array.isArray(value);  
}  
export function isEntry(value) {  
  if (!isRecord(value))  
    return false;  
  return (typeof value.id === 'string' &&  
    typeof value.title === 'string' &&  
    typeof value.added === 'string');  
}  
export function latest(entries) {
```

TypeScript never changes the runtime behaviour of JavaScript code.

THE SAME FILE, WITH ONE WORD ADDED

a real localStorage round trip, and the compiler's real answer

reloadopen

A DATE IN STATE, AND WHAT A RELOAD DOES TO IT

add an item

save to localStorage

load it back

reset

THE COLLECTION, DRAWN

nothing in the collection yet

THE SAME CODE, TYPED — REAL tsc OUTPUT

```
8
9 function addItem(title: string): Item {
10   return {
11     id: crypto.randomUUID(),
12     title,
13     created: new Date(), // <- the week-11 bug, caught here instead of after a reload
14   };

```

TS2322: Type 'Date' is not assignable to type 'string'.

1 error — line 13

The right-hand panel is loaded from tsc-output.json, captured by running the compiler over tsc-cases/01-date-in-state.ts. Nothing on this page quotes a compiler message from memory.

▶ Press **ADD** twice — it works. Then **SAVE**, then **LOAD**.

5

← TAUGHT · 4

DESCRIBING A SHAPE: `interface` OR `type`

```
1 interface ShelfEntry {  
2   readonly id: string; // fixed once set: next slide  
3   title: string;  
4   added: string;       // an ISO date as text, never a Date  
5 }  
6  
7 type LookupKind = 'offline' | 'status' | 'shape';
```

`interface`

A **shape**: which fields one object has, and what each holds. The default for a thing your code passes around.

`type`

A name for **anything else**. Here, a **union**: the value is one of exactly these words, and any other word is a compile error.

THE RULE THAT SETTLES IT

A union **cannot be written as an interface at all**. Where both would work, pick one and stay consistent.

YOU HAVE BEEN WRITING UNIONS SINCE WEEK 10

TWO BOOLEANS

```
1 let isLoading = false;
2 let isEmpty = false;
3 // four combinations, two of them nonsense
```

ONE FIELD, FOUR VALUES

```
1 type RequestStatus =
2   'idle' | 'loading' | 'done' | 'error';
3 let status: RequestStatus = 'idle';
```

Week 11's argument was that a field with four values cannot be in two of them. Today the compiler enforces which four.

THE ONES YOU ALREADY HAVE, NAMED

```
1 string number boolean
2 null undefined
3 symbol bigint
4
5 // any stop checking
6 // unknown nobody checked yet
7
8 const tags: string[] = []; // any length
9 const pair: [number, number] = [0, 0]; // two
10
11 // worth the ceremony:
12 const [y, m, d] = iso.split('-');
13 // ^ string[], so d may be undefined
```

1 Seven primitives, and you know all of them
— this is week 8's list with a compiler attached.

2 `T[]` is “any number of `T`”. `[A, B]` is
exactly two, and they mean different things.

Under `noUncheckedIndexedAccess`, `items[0]` is
3 `T | undefined` —
because it is.

THREE SMALL MARKS: `readonly`, `| null`, `?`

```
1 interface Book {  
2   readonly id: string;  
3   title: string;  
4   author: string;  
5   year: number | null;  
6   pages: number;  
7 }
```

1 `readonly id` — **set once**. Assigning to it later is a compile error. One word, no cost.

2 `year: number | null` — **the key is there and holds nothing**. A book of unknown year still has a year slot.

3 Not in `Book`, for contrast: `year?: number` would mean **optional** — the key may be missing altogether.

► `| null` for a field you sort or filter by: a book with no year has to land somewhere, **IN BOTH DIRECTIONS**.

THE BUILD STEP: `ts/` IN, `js/` OUT

TSCONFIG.JSON · ABRIDGED — THE REAL FILE ADDS FIVE MORE LINES

```
1 {
2   "compilerOptions": {
3     "target": "ES2022",
4     "module": "ES2022",
5     "lib": ["ES2022", "DOM", "DOM.Iterable"],
6     "strict": true,
7     "noEmitOnError": true,
8     "rootDir": "ts",
9     "outDir": "js",
10    "declaration": true
11  },
12  "include": ["ts/**/*.ts"]
13 }
```

1 `strict` — the full set of checks. **Without it, none of today happens.**

2 `noEmitOnError` — a type error writes **no output**, so the page never loads a half-checked file.

3 `rootDir / outDir` — sources in `ts/`, output in `js/`, at the same relative path.

4 `declaration` — a `.d.ts` beside each output: types only, no code. It gives your plain JavaScript files the types **without migrating them.**

► `npx tsc -p tsconfig.json` — `npx` runs the installed `tsc`, `-p` names the settings file. That is the build.

noEmitOnError: TWO ERRORS, AND NOT ONE FILE WRITTEN

CYCLE 1, STEP 2 — TS/STORAGE.TS RENAMED, 2 ERRORS	EXIT CODE	WHAT IS IN JS/ AFTERWARDS
<code>npx tsc -p tsconfig.json</code> — the assignment's file, <code>"noEmitOnError": true</code>	1	nothing new — the browser still loads the old, hand-written <code>storage.js</code>
the same, with <code>--noEmitOnError false</code>	2	<code>storage.js</code> , <code>types.js</code> and both <code>.d.ts</code> written anyway — from code the compiler just rejected

A type error must not leave a half-checked file on disk for the page to load. With the option, a red run changes nothing you can see.

WHERE `npx tsc` FINDS THE COMPILER

1 `npx tsc` looks for a `tsc` it already has: the folder's own `node_modules`, then your `PATH` — which is where a global install puts it.

The student repo has no `package.json` and no `node_modules`

2 , so the one it finds is the global one: `npm install --global typescript@5.9.3`, once, before this week (`pre-class/` checks it).

3 Measured on a folder with nothing in it: `npx --offline tsc --version` → `Version 5.9.3`, from the global install, with no network.

► If `npx` ever offers to **DOWNLOAD** a package called `tsc` — say no. On npm, `tsc` is version 2.0.4, “a deprecated release of the TypeScript compiler”. Install `typescript` instead.

TYPES FIRST, THEN THE ONE MODULE WHERE OUTSIDE VALUES COME IN

- 1 Types first.**
`ts/types.ts` — the entity, the unions, the guard.
- 2 One module, and the right one — the boundary:**
where a value you did not write comes in — `storage.js` in cycle 1, `api.js` in cycle 3, the pantry's `api.js` in the assignment.
- Rename it and compile.**
3 The compiler emits `js/storage.js` at the same path, so **nothing that imports it changes.**
- 4 Fix the first error only**
, and run again. Read the count out loud.
- 5 Stop.**
The other modules stay JavaScript, and get their types from the emitted `.d.ts` anyway.

as IS A DECLARATION, NOT A CHECK

COMPILES. CHECKS NOTHING.

```
1 const parsed = JSON.parse(raw) as ShelfEntry;
2 parsed.title.trim(); // compiles
```

ONE WORD LONGER: TS18046, 'PARSED' IS OF TYPE 'UNKNOWN'

```
1 const parsed: unknown = JSON.parse(raw);
2 parsed.title.trim(); // TS18046
```

WHAT YOU WRITE

WHAT IT MEANS

WHAT RUNS

: any

stop checking, here and downstream —
JSON.parse and res.json() hand you one

nothing

: unknown

nobody has checked this yet

nothing — **but you cannot touch it until something does**

as T

I promise it is a T

nothing

value is T

here is the proof (a **guard**)

the function body

as AND readonly, AS THE COMPILER REPORTS THEM

two captured cases – every message came out of `tsc --strict`

[↺ reload](#)[open ↗](#)

WHAT THE COMPILER SAYS ABOUT `as`, AND ABOUT `readonly`

`as` — A PROMISE, CHECKED ONLY AGAINST ABSURDITY

```
1 interface Suggestion {
2   title: string;
3   url: string;
4 }
5
6 // `as` compiles. It checks nothing, and it is not a conversion.
7 function trustIt(payload: unknown): Suggestion {
8   return payload as Suggestion; // compiles happily. `payload` may be a 404 document.
9 }
10
11 // The compiler DOES stop an assertion between unrelated types – which is why the
12 // "fix" people reach for next is `as unknown as T`, and why that is a smell.
13 function trustHarder(n: number): Suggestion {
14   return n as Suggestion;
15 }
```

TS2352: Conversion of type 'number' to type 'Suggestion' may be a mistake because neither type sufficiently overlaps with the other. If this was intentional, convert the expression to 'unknown' first.

0 diagnostics on line 8 (unknown → Suggestion) · line 14 TS2352 (number → Suggestion) — and the message itself suggests going through unknown first.

`readonly` — ONE WORD, AND CHANGING A SHARED OBJECT STOPS COMPILING

▶ unknown as T: **NO DIAGNOSTIC**. number as T: TS2352 — which says to go through unknown.

THE ONLY HONEST BRIDGE OUT OF `unknown`

```
1 function isRecord(value: unknown): value is Record<string, unknown> {  
2   return typeof value === 'object' && value !== null && !Array.isArray(value);  
3 }  
4 export function isEntry(value: unknown): value is ShelfEntry {  
5   if (!isRecord(value)) return false;  
6   return (  
7     typeof value.id === 'string' &&  
8     typeof value.title === 'string' &&  
9     typeof value.added === 'string'  
10  );  
11 }
```

`: boolean`

“the answer is yes”

`: value is ShelfEntry`

“... and from here on you may treat it as one” — a **guard**.

`parsed.filter(isEntry)` is then a `ShelfEntry[]`.

isRecord FIRST

`typeof null` and `typeof []` are both `'object'`, so a real object takes **three** checks.

`Record<string, unknown>`: an object with any string keys, values not yet checked.

FOUR WAYS TO ASK WHAT A VALUE IS

every check, against every value – computed

↺ reload

open ↗

FOUR WAYS TO ASK WHAT A VALUE IS

run every check against every value

COMPUTED, NOT WRITTEN OUT

AND THE UNION, THROUGH `tsc --strict`

```
1 type ErrorKind = 'offline' | 'timeout' | 'status' | 'shape';
2
3 function sentence(kind: ErrorKind): string {
4   if (kind === 'offline') return 'no route out';
5   if (kind === 'timeout') return 'took too long';
6   if (kind === 'status') return 'they answered, badly';
7   if (kind === 'shape') return 'not what we expected';
8   return kind; // `never` here: every arm is covered, so nothing reaches this line
9 }
10
11 function typo(kind: ErrorKind): string {
12   if (kind === 'timout') return 'took too long'; // <- one letter
13   return '';
14 }
15
```

TS2367: This comparison appears to be unintentional because the types 'ErrorKind' and '"timout"' have no overlap.

line 12 — TS2367. One letter, found before it ran.

► Read the `null` row. **THE FIRST COLUMN SAYS IT IS AN OBJECT.**

READ THE COUNT OUT LOUD: 2, THEN 4, THEN 0

1 · RENAME → 2

`ts/storage.ts` is `js/storage.js`, word for word. Two errors, both `TS7006`: a parameter with no type — nobody has called the function yet, so the compiler cannot **infer** it. **Nothing is written to `js/`.**

2 · FIX THE FIRST → 4

`value: unknown`. Now `Property 'id' does not exist on type 'object'`, three times. **The count went up** — the compiler is asking what the JavaScript assumed.

3 · THE GUARD → 0

`isRecord` first, then three `typeof` checks, in `isEntry(value: unknown): value is ShelfEntry`. Zero, and four files in `js/`.

► **THE RULE:** write the type on parameters, on return types (the `: Shelf` after the parentheses) and on anything from outside; the compiler infers the rest. And **FORTY ERRORS ARE NOT FORTY PROBLEMS** — usually one, in forty places.

MOST ANNOTATIONS YOU WRITE ARE ALREADY KNOWN

NOISE

```
1 const name: string = 'πηλ';  
2 const count: number = 3;  
3 const items: string[] = ['a', 'b'];
```

THE SAME THING

```
1 const name = 'πηλ'; // string  
2 const count = 3; // number  
3 const items = ['a', 'b']; // string[]  
4  
5 // here it cannot be inferred:  
6 function shout(word: string): string {  
7     return word + '!';  
8 }
```

Annotate parameters, return types, and anything from outside. Let inference do the rest.

CYCLE 1 · 28 MIN

RENAME, COMPILE, AND GUARD THE BOUNDARY

The class lending library, four JavaScript modules and the assignment's own `tsconfig.json`. One module — `storage.js`, where the disk comes in — moves to `ts/storage.ts`. You read the error count out loud at every run, and the screen ends exactly as it started.

1 I DO

15 MIN

8 on the slides up to here, 7 in the editor · live-code/01-rename-and-compile · types.ts first (copy this) · rename, run: 2, then 4 · the guard value is `ShelfEntry` ·
`'./types'` (wrong) ·
`'./types.js'`

2 WE DO

4 MIN

steps 1 to 5 on your copy (steps 1, 2 and 4 are **copy this**; run, say the count) · compare with checkpoint 1

3 YOU DO

7 MIN

the second guard: `isBook` — strict on what the list breaks without, tolerant on a missing year

4 DEBRIEF

2 MIN

zero errors and a blank page · the count that went up · ↗ the deeper slides

YOU DO · AI-FREE

A VALUE IMPORT KEEPS ITS PATH

```
1 import { isEntry } from './types.js'; // a function: stays in js/storage.js
2 import type { ShelfEntry } from './types.js'; // a type: erased, gone from the output
```

WHAT TS/STORAGE.TS SAYS	NPX TSC -P TSCONFIG.JSON	THE BROWSER
from './types' — what the editor's auto-import writes	0 errors	blank page — 404 on js/types
from './types.ts'	TS5097 — refused	—
from './types.js'	0 errors	loads

The path in an `import` is the address the browser will ask for — and the browser loads `js/`.

CHECKPOINT 1

YOUR SCREEN SHOULD LOOK LIKE THIS – EXACTLY LIKE IT DID BEFORE

ספריית ההשאלה

המדף שלי

6 ספרים בקטלוג · שורה אחת לא נקראה

שמרתי לקריאה

המדף ריק. הוסף ספר מהקטלוג.

הקטלוג

חיפוש בקטלוג

אור בחלון

נועה ברק · 2019 · כ-6 שעות קריאה

+ למדף

גשר על הירקון

דן אלמוג · 2008 · כ-4 שעות קריאה

+ למדף

הים של סבתא

רות גל · 2021 · כ-2 שעות קריאה

+ למדף

כוכבים בצהריים

מיכל שחר · שנה לא ידועה · כ-9 שעות קריאה

+ למדף

מפה בלי צפון

יונתן לב · 2015 · כ-3 שעות קריאה

+ למדף

שבע דלתות

עמרי צור · 2003 · כ-11 שעות קריאה

+ למדף

Compare — six books and “one row could not be read”, the same as `start/`. Then: `ls js/` shows `types.js` and `storage.d.ts`; `js/types.js` has two functions and no `interface`; both imports in `storage.ts` end in `.js`.

WHY `tsc` ACCEPTS `'./types'`, AND THE BROWSER DOES NOT

The assignment's `tsconfig` says

- 1 `"moduleResolution": "bundler"`: resolve imports the way a bundler would — and a bundler finds `./types.ts` for `'./types'`.

- 2 **The output keeps the specifier exactly as written.**
`tsc` does not rewrite `'./types'` to `'./types.js'`.

- 3 There is no bundler here. The browser asks the server for `js/types`, gets a `404`, and the whole module graph stops — nothing that imports it runs.

```
// js/storage.js, as emitted:  
import { isEntry } from './types';  
// → GET /js/types → 404  
  
// no `import type` line at all:  
// types are erased, so a type-only  
// import never breaks anything
```

► **THE RULE THAT NEVER FAILS:** in a `.ts` file, write the path of the **OUTPUT** — `'./types.js'`. `'./types.ts'` is refused (TS5097).

THE CASTING YOU WILL ACTUALLY NEED

four real diagnostics, the helper, and two error messages compared

↺ reload

open ↗

ELEMENT OR NULL, AND NEITHER ONE IS WHAT YOU WANTED

A FORM TO QUERY

search

- first
- second

read the box, both ways

a selector that matches nothing

RAW `querySelector`

`$<T>()` — ONE ASSERTION, IN ONE PLACE

AND WHAT `tsc --strict` SAYS — THE RAW VERSION, THE HELPER, THE EVENT

```
1 // `querySelector` returns `Element | null`, and both halves are in the way.
2 function readBox() {
3   const box = document.querySelector('#query');
4   return box.value; // two errors on one line
5 }
6
7 // One generic helper, one assertion, in a place you can find again.
8 function $<T extends Element = Element>(selector: string): T {
9   const found = document.querySelector(selector);
10  if (found === null) throw new Error(`no element matches ${selector}`);
```

► A **CAST** is an `as`: you tell the compiler what a value is. Press **A SELECTOR THAT MATCHES NOTHING**. Only one of the two messages names the typo.

ONE GENERIC, READ ALOUD

```
1 export function $<T extends Element = Element>(
2   selector: string,
3   scope: ParentNode = document,
4 ): T {
5   const found = scope.querySelector(selector);
6   if (found === null) throw new Error(`nothing on the page matches ${selector}`);
7   return found as T;
8 }
9
10 $<HTMLInputElement>('#q').value; // compiles: T is HTMLInputElement
11 $('#status').textContent;        // T is Element, the default
```

T

a parameter whose value is a **type**
— the caller picks it

extends Element

some kind of element — **not anything**: `$<number>` does not compile

= Element

the default, so `$('main')` works with no type argument at all

THE GENERIC THAT PROMISES WHAT IT NEVER CHECKS

two signatures, one file, and what `tsc --strict` said about both

↺ reload

open ↗

TWO SIGNATURES THE COMPILER CANNOT TELL APART

the endpoint answers:

`getJSON<T>(url): Promise<T>`

`getChecked<T>(url, guard): Promise<T>`

BOTH OF THEM, THROUGH `tsc --strict`

`tsc --strict`, over both versions: 0 errors.

```
1 interface Suggestion {
2   title: string;
3 }
4
5 // THE SIGNATURE PEOPLE WRITE FIRST. It compiles, and it is a lie: there is no `T` at
6 // runtime, so nothing here checks anything. Every caller gets a fully-typed object that
7 // may be an error document.
8 async function getJson<T>(url: string): Promise<T> {
9   const res = await fetch(url);
10  return res.json();
11 }
12
13 // THE HONEST ONE. The caller supplies the type AND the proof.
14 async function getChecked<T>(url: string, guard: (v: unknown) => v is T): Promise<T> {
15   const res = await fetch(url);
16   const body: unknown = await res.json();
17   if (!guard(body)) throw new Error('the response is not what we asked for');
```

► **VOTE FIRST.** Which of the two is the wrong one? (`getJSON<T>` is the signature every assistant writes first.)

CYCLE 2 · 25 MIN

TYPE THE DOM: ONE HONEST GENERIC

`querySelector` returns `Element | null`, and both halves are in the way. One generic function makes the assertion **once, with a name, in a place you can find** — and throws with the selector in the message when the element is not there. `app.js` stays JavaScript and imports it.

1 I DO

10 MIN

3 on the slides up to here, 7 in the editor · live-code/02-type-the-dom · `querySelector(...).value`: 2 errors on one line · `$` with a bare `as T` (wrong) · throw with the selector · the typo, by name

2 WE DO

5 MIN

`$`, `inputText`, the import in `app.js` · compare with checkpoint 2

3 YOU DO

8 MIN

the event half: `rowOf(event)` with `instanceof Element`, wired into both click handlers

4 DEBRIEF

2 MIN

forty as `HTMLInputElement` · the helper that returns null quietly · ↗ the deeper slides

YOU DO · AI AS PARTNER

AND THE EVENT, WHICH IS THE SAME TRICK

NEITHER OF THESE IS RIGHT

```
1 // does not compile:
2 const row = event.target.closest('li');
3
4 // compiles, and lies:
5 const row = (event.target as HTMLElement)
6   .closest('li');
```

ONE LINE, AND IT IS TRUE AT RUNTIME

```
1 if (!(event.target instanceof Element))
2   return null;
3 // narrowed from here on:
4 return event.target
5   .closest<HTMLElement>('li[data-id]');
```

instanceof asks, while the page runs, whether a value was built by a given kind of object — and the compiler understands the answer. That is **value is T** wearing a different hat.

CHECKPOINT 2

YOUR SCREEN SHOULD LOOK LIKE THIS – STILL

ספריית ההשאלה

המדף שלי

6 ספרים בקטלוג · שורה אחת לא נקראה

שמרתי לקריאה

המדף ריק. הוסף ספר מהקטלוג.

הקטלוג

חיפוש בקטלוג

אור בחלון

נועה ברק · 2019 · כ-6 שעות קריאה

למדף +

גשר על הירקון

דן אלמוג · 2008 · כ-4 שעות קריאה

למדף +

הים של סבתא

רות גל · 2021 · כ-2 שעות קריאה

למדף +

כוכבים בצהריים

מיכל שחר · שנה לא ידועה · כ-9 שעות קריאה

למדף +

מפה בלי צפון

יונתן לב · 2015 · כ-3 שעות קריאה

למדף +

שבע דלתות

עמרי צור · 2003 · כ-11 שעות קריאה

למדף +

Compare — the same screen. Then: type the first word of one title in the search — one row; `js/dom.js` has no `as` and no `<T`; `app.js` has no `document.querySelector ; $('#nope')` in the console throws, and the message says `#nope`.

THE DOM LIBRARY ALREADY SPEAKS THE SAME GENERIC

```
1 // lib.dom.d.ts – interface ParentNode
2 querySelector<E extends Element = Element>(selectors: string): E | null;
3
4 // lib.dom.d.ts – interface Element
5 closest<K extends keyof HTMLElementTagNameMap>(selector: K): HTMLElementTagNameMap[K] | null;
6 closest<E extends Element = Element>(selectors: string): E | null;
```

THE SAME THREE PARTS

`E`, `extends Element`,
`= Element` — letter for letter the
shape of cycle 2's `$`.

A TAG NAME IS ENOUGH

`closest('li')` is already
`HTMLLIElement | null`. A
selector like `'li[data-id]'` is
not a tag name, so it falls to the
generic — hence
`closest<HTMLElement>(…)` in
row 0f.

WHAT `$` ADDS

Only the `| null` removed, and a
`throw` that names the selector.
That is the whole reason to write it.

CYCLE 3 · 25 MIN · AI

MIGRATE `api.js` — AND READ WHAT IT SILENCED

Prompt 1 is what almost everyone asks: “move it to TypeScript, make it compile”. What returns compiles with zero errors — and silences the compiler three ways (four lines), one of which deletes the guard. You find all three with the browser, `grep`, and `tsc` with the silencing removed.

1 I DO

8 MIN

3 on this slide and the prompt slide, 5 in the chat · ai-lab/03-migrate-api · P1 live · paste it · `tsc : 0` · reload

2 WE DO

6 MIN

browser, `grep`, then `tsc` without the `@ts-ignore : 1`, then 0 (after the missing field), then 2 (after any becomes unknown) · type the missing field yourself

3 YOU DO

9 MIN

your own P2 with four checkable rules · review before you run · one review line, in the exemplar’s form

4 DEBRIEF

2 MIN

what the `@ts-ignore` was hiding · the declaration graded this week · ↗ the deeper slides

YOU DO · AI AS PARTNER

PROMPT 1, AND WHAT CAME BACK

IT COMPILES. FIND THE THREE KINDS OF SWITCH-OFF — FOUR LINES IN ALL.

THE PROMPT, WORD FOR WORD

Move js/api.js to TypeScript. The new file: ts/api.ts.
It has to compile with `npx tsc -p tsconfig.json` — the tsconfig and ts/types.ts are below.
Keep the behaviour as it is.

```
<js/api.js · ts/types.ts · tsconfig.json>
```

THE ANSWER, CUT TO THE LINES THAT MATTER (WHOLE: TRANSCRIPT.MD)

```
1  constructor(kind: LookupKind, messageHe: string) {  
2    super(messageHe);  
3    // @ts-ignore — kind is assigned at runtime  
4    this.kind = kind;  
5  }  
6  ...  
7  } catch (error: any) {  
8    ...  
9    const payload: any = await res.json();  
10   // The response is typed now, so every entry is a Book.  
11   const books = payload.books as Book[];  
12   return { books, skipped: 0 };
```

► `tsc`: zero errors. Reload: **“7 BOOKS IN THE CATALOGUE”** — there are six — and an empty list.

WHICH LINE LET THE ROW WITHOUT A TITLE THROUGH?

► `any` — stop checking. `as Book[]` — trust me. `// @ts-ignore` — do not report the errors on the next line.

WHAT `tsc` SAYS WHEN YOU STOP SILENCING IT

TS/API.TS	NPX TSC -P TSCONFIG.JSON	WHAT IT POINTS AT
as Claude returned it	0	nothing — that is what the three silencers are for, and <code>res.json()</code> is <code>Promise<any></code> in the DOM's own types
without the <code>@ts-ignore</code>	1 — TS2339	Property 'kind' does not exist on type 'LookupError': the field was never declared
... you type <code>readonly kind</code> (back to 0), then both <code>any</code> become <code>unknown</code>	2 — TS18046 ×2	'payload' is of type 'unknown' — exactly the two lines that need a guard

```
1 export class LookupError extends Error {
2   readonly kind: LookupKind;           // you type this — and the 1 is 0
3   ...
4   interface CatalogueFile { books?: unknown } // their schema: declared in api.ts, no export
5   const file: CatalogueFile = payload;       // after isRecord(payload): no cast needed
6   ...
7   async function lookup(query: string,        // type the whole pattern; ? = may be left out
8     { signal, retries = 1 }: { signal?: AbortSignal; retries?: number } = {}) {
```

The count the compiler prints when nobody silences it is the work list.

CYCLE 3 · ON SCREEN BEFORE ANYONE WRITES A REVIEW

A REVIEW OF MACHINE-WRITTEN CODE THAT SOMEBODY CAN CHECK

THE GOOD ONE

The claim and the conclusion are two lines. What the explanation said; what actually runs; and **the code that proved it, with what it printed.** The two functions that were right are left alone.

THE BAD ONE

“I read the code and understood the order can change.”
No run, no output — and a claim about `Set` that is itself wrong.

► **AI-EXEMPLARS/EXEMPLARS/AI-REVIEW-JS-GOOD.MD**, then **AI-REVIEW-JS-BAD.MD**, in your course materials. Assignment 12 asks for the same shape in `PROMPTS.md`: what it wrote, what you changed, **AND WHY**.

CHECKPOINT 3

THREE QUESTIONS, THREE YESES

You ran the compiler

1 after deleting the `@ts-ignore`

, read `TS2339`, and fixed it with a declaration — not with another silencer.

Every rule in your P2 is something `grep`, `tsc` or the browser can check, and the answer passed all four

2 before you ran it

.

Your review line says

3 what you saw

— 7 where there are 6, a console error, an error code.

GRADED FOR THE FIRST TIME THIS WEEK

THE DECLARATION: `AI_USAGE.md`, GOOD AND BAD

THE GOOD ONE

By file. “Can I explain every line?” is **not “yes” on every row** — where it is “partly”, it says which lines, and what you will do before week 13.

THE BAD ONE

“About 40%.” “I understand all the code: yes.” Honest, and worth nothing — to the grader, and to you a week before the exam.

► **AI-EXEMPLARS/EXEMPLARS/AI-DECLARATION-GOOD.MD**, then **AI-DECLARATION-BAD.MD**. Due with the project, this week — and the exam is where it is checked.

► And the project’s `README.md`: **AI-EXEMPLARS/EXEMPLARS/README-GOOD.MD**, then **README-BAD.MD** — what it is, how to run it, which API, and **WHAT YOU DID NOT BUILD**.

A FUNCTION'S TYPE IS ITS CONTRACT

```
1  async function searchTitles(  
2    query: string,  
3    { signal, retries = 1 }: SearchOptions = {},  
4  ): Promise<Suggestion[]> { /* ... */ }  
5  
6  // the return type is not decoration. Delete it and this still compiles:  
7  function toSuggestion(page: unknown) {  
8    if (!ok(page)) return null;  
9    return { title, note, url, index }; // caller now gets `| null` by accident  
10 }
```

WRITE THE RETURN TYPE

A change inside the function that changes what it gives back is caught **in the function**, not three files away.

AN `async` FUNCTION

always returns a `Promise`. So the annotation is `Promise<T>`, never `T`.

A DEFAULT IS NOT OPTIONAL

`retries = 1` means the caller may omit it. Inside, it is a `number`, never `undefined`.

any AND unknown AT THE ONE BOUNDARY THAT MATTERS

the same body, two readers – and what tsc said

reload

open ↗

THE SAME BODY, TWO READERS

body: a real answer

read it both ways

THE any READER — NO ANNOTATION

```
const payload = await res.json();
return payload.query.pages[0].title;
```

THE unknown READER — ONE WORD

```
const payload: unknown = await res.json();
if (!isPayload(payload)) return 'nothing to show';
return payload.query.pages[0].title;
```

AND WHAT tsc SAID ABOUT THOSE TWO LINES

```
1 // `res.json()` is typed `Promise<any>` in the DOM lib. That is the whole problem.
2 async function withAny(res: Response) {
3   const payload = await res.json();
4   return payload.query.pages[0].title.toUpperCase(); // no complaint. On a 404 body.
5 }
6
7 async function withUnknown(res: Response) {
8   const payload: unknown = await res.json();
9   return payload.query.pages[0].title.toUpperCase(); // <- the compiler asks
10  }
11
12 // INTENTIONALLY UNCOMPILABLE – this file is a teaching example. Its diagnostics are
13 // captured by demos/capture-tsc.mjs and shown on the slides; tools/gates.mjs inverts the
```

TS18046: 'payload' is of type 'unknown'.

▶ Run the good body first. Then the **404**, which is valid JSON.

BREAK

10 MINUTES

Back at 95 for one short cycle — the last one before the exam, and AI-free. At 115, the surgery: no AI, no internet, alone, forty minutes.

THE EXAM'S THIRD VERB

REFACTORING

A third of the exam, and the only one of the three verbs whose success is invisible.

A refactor is a change to the shape of code that does not change its behaviour.

```
const { getState, selectVisible } = await import('./js/state.js'); // in the console of your project
const items = [{ id: 'b', title: 'Beta' }, { id: 'a', title: 'Alpha' }, { id: 'c', title: 'Gamma' }];
const ids = (patch) => selectVisible({ ...getState(), items, ...patch }).map((i) => i.id).join(',');
console.assert(ids({ query: 'ta' }) === 'b', 'the query filters');
console.assert(ids({ query: 'zzz' }) === '', 'no match is an empty list');
console.assert(items.map((i) => i.id).join(',') === 'b,a,c', 'state.items was not reordered');
```

► The same proof, written down: **SILENT BEFORE THE REFACTOR, SILENT AFTER**. A red line in the console is a behaviour that changed — and a test you wrote yourself.

“IDENTICAL”, MEASURED INSTEAD OF PROMISED

four versions, 240 inputs, one question

reloadopen

FOUR VERSIONS, 240 INPUTS, ONE QUESTION

run every version over every inputshow the first difference

DIFFERENCES FROM THE ORIGINAL

V0 — THE ORIGINAL

```
function v0(loan, today) {
  if (loan !== null) {
    if (loan.out) {
      if (loan.due !== null) {
        if (loan.due < today) {
          const [y, m, d] = loan.due.split('-');
          return 'late since ' + d + '/' + m + '/' + y;
        } else {
          const [y, m, d] = loan.due.split('-');
          return 'back on ' + d + '/' + m + '/' + y;
        }
      } else {
        return 'out, no date';
      }
    } else {
      return 'in';
    }
  } else {
    return '-';
  }
}
```

V3 — THE ONE THAT IS NOT A REFACTOR

```
function v3(loan, today) {
  if (!loan?.out) return loan ? 'in' : '';
  if (!loan.due) return 'out, no date';
  const [y, m, d] = loan.due.split('-');
  return `${loan.due < today ? 'late since' : 'back on'} ${d}/${m}/${y}`;
}
```

▶ Three zeros and one number. **GUESS WHICH VERSION BEFORE YOU LOOK.**

FOUR SUBMISSIONS, GRADED IN FRONT OF YOU

the same task, four plausible answers, live scoring

↺ reloadopen ↗

THE TASK: REMOVE THE DUPLICATION. BEHAVIOUR IDENTICAL.

THE FIVE CRITERIA, AND WHICH TWO A MACHINE CAN ANSWER

1. Behaviour is identical — every input, every output · a machine can answer this

2. The thing that was written twice is now written once · a machine can answer this

3. The names say what, not how or where · a person must

4. The change is the size of the task and no larger · a person must

5. A future change that used to touch N places now touches one · a person must

grade all four

why did D lose the most?

▶ **RANK THEM BEFORE YOU PRESS GRADE.** Then look at D.

HOW TO REFACTOR SAFELY WHEN YOU HAVE NO TESTS

1 Write down what must not change
before you start — three behaviours, and how you will check each.

2 One move at a time.
Extract, or rename — not both.

Verify after each.
3 Reload, click the thing, compare the screen.
Ten seconds.

4 Commit after each.
A commit per move is how you find out which move broke it.

5 If you cannot check it, do not do it.
Cut the scope until you can.

► The first step is the one people skip, and it is the one the exam sheet asks for **IN WRITING**.

THE CATALOGUE — SEVEN MOVES, AND YOU NEED ALL OF THEM

THE MOVE	THE SMELL (THE SIGN IN THE CODE) THAT CALLS FOR IT
extract function	a comment explaining what the next five lines do
rename for intent	<code>data, temp, handleClick2</code>
remove duplication	the same three lines, twice, with one word different
guard clauses over nesting	the real work is four indents from the margin
split by responsibility	a function whose name needs the word “and”
name the magic value	<code>86400000, '-', 0.6</code>
simplify the conditional	<code>if (x === true) return true; else return false;</code>

EXTRACT, AND RENAME — THE TWO THAT PAY IMMEDIATELY

A COMMENT DOING A FUNCTION'S JOB

```
1 function renderRow(item) {  
2   // turn the ISO date into dd/mm/yyyy  
3   const [y, m, d] = item.due.split('-');  
4   const text = `${d}/${m}/${y}`;  
5   const data = item.number * 0.6; // ???  
6   return `${item.title} - ${text} - ${data}`;  
7 }
```

THE COMMENT BECAME THE NAME

```
1 const DISCOUNT = 0.6;  
2 const formatDate = (iso) =>  
3   iso.split('-').reverse().join('/');  
4  
5 function renderRow(item) {  
6   const cost = item.number * DISCOUNT;  
7   return `${item.title} - ${formatDate(item.due)}`;  
8 }
```

A comment explaining the next lines is a function asking to be born.

SPLIT BY RESPONSIBILITY, AND SIMPLIFY THE CONDITIONAL

ITS NAME NEEDS THE WORD “AND”

```
1 function validateAndSave(item) {  
2   if (item.title === '') return false;  
3   save(item);  
4   if (item.done === true) {  
5     return true;  
6   } else {  
7     return false;  
8   }  
9 }
```

TWO JOBS, TWO FUNCTIONS

```
1 const isValid = (item) => item.title !== '';  
2  
3 function saveItem(item) {  
4   if (!isValid(item)) return false;  
5   save(item);  
6   return item.done;  
7 }
```

► `if (x === true) return true; else return false;` is `return x;` — and it is in more code than anybody admits.

GUARD CLAUSES: EVERY WAY OUT AT THE TOP

THE WORK IS FOUR INDENTS IN

```
1 function label(loan) {  
2   if (loan !== null) {  
3     if (loan.out) {  
4       if (loan.due !== null) {  
5         return format(loan.due);  
6       } else { return 'out, no date'; }  
7     } else { return 'in'; }  
8   } else { return '-'; }  
9 }
```

EVERY EXIT AT THE TOP

```
1 function label(loan) {  
2   if (loan === null) return '-';  
3   if (!loan.out) return 'in';  
4   if (loan.due === null) return 'out, no date';  
5   return format(loan.due);  
6 }
```

The last line is the point of the function, and now you can see it. An `else` after a `return` is always unnecessary — the first thing to search for.

CYCLE 4 · 20 MIN

ONE REFACTOR, THREE COMMITS

The last cycle before the surgery, and the exam's third verb. `render.js` of the library, three smells: a number with no name, a calculation written twice, a function three levels deep. One move per commit, the screen compared after every one — and one “improvement” that is not a refactor.

1 I DO

8 MIN

3.5 on the slides up to here, 4.5 in the editor · live-code/04-three-commits · three lines on the board first · name the constant · extract, with `Math.round` (wrong) · `Math.ceil`

2 WE DO

5 MIN

screenshot first, then commits 1 and 2 · compare with checkpoint 4

3 YOU DO

5 MIN

commit 3: guard clauses in `shelfNote` — every exit at the top, no `else`

4 DEBRIEF

2 MIN

“and I improved it on the way” · ↗
the deeper slides

YOU DO · AI-FREE

CHECKPOINT 4

THE SAME SCREEN AS YOUR SCREENSHOT, AND TWO COMMITS

ספריית ההשאלה

המדף שלי

6 ספרים בקטלוג · שורה אחת לא נקראה

שמרתי לקריאה

המדף ריק. הוסף ספר מהקטלוג.

הקטלוג

חיפוש בקטלוג

אור בחלון

נועה ברק · 2019 · כ-6 שעות קריאה

+ למדף

גשר על הירקון

דן אלמוג · 2008 · כ-4 שעות קריאה

+ למדף

הים של סבתא

רות גל · 2021 · כ-2 שעות קריאה

+ למדף

כוכבים בצהריים

מיכל שחר · שנה לא ידועה · כ-9 שעות קריאה

+ למדף

מפה בלי צפון

יונתן לב · 2015 · כ-3 שעות קריאה

+ למדף

שבע דלתות

עמרי צור · 2003 · כ-11 שעות קריאה

+ למדף

Compare — with the screenshot you took before the first move, not with this picture alone: every reading time the same. Then: `grep -c "40" js/render.js` is 1; `Math.ceil` appears once; `git log --oneline` shows two commits whose names say what moved.

SURGERY · 115–165

FORTY MINUTES ALONE, EIGHT TOGETHER, THEN TEN ON THE MIGRATION

117–157 · ALONE

The full surgery. One repair, one addition, one restructure, on an application you have not seen. **No AI, no internet, no neighbour.**

Two versions in the room — check that your sheet and your folder agree.

157–165 · TOGETHER

The walkthrough, straight away — **and the grader running on screen,** so you see exactly how the automatic points are awarded.

165–175 · THE ASSIGNMENT

The spec, first and by hand, and its commit. The migration itself — typed today on the library — **at home, before lesson 13.**

The three sentences on that paper are word for word the three sentences on the exam.

GOOD, VERSUS MERELY DIFFERENT

#	CRITERION	WHO CAN ANSWER IT
1	behaviour is identical — every input, every output	a machine
2	the thing written twice is now written once	a machine
3	the names say what , not how or where	a person
4	the change is the size of the task, and no larger	a person
5	a future change that touched N places now touches one	a person

These five are on the exam's answer key, in this order. They are not a secret.

ASSIGNMENT · 165–175

THE SPEC COMES FIRST: FIVE SECTIONS, WRITTEN BEFORE ANY TOOL

#	SECTION OF <code>SPECS/TS-MIGRATION.MD</code>	WHAT IT MAKES YOU DECIDE	WHERE YOU DID IT TODAY
1	what I migrate, and what I do not	one file; what stays JavaScript this week, and why	cycle 1: one module, the rest stay
2	the types the code needs	<code>interface</code> or <code>type</code> for each, and why that one	cycle 1: <code>ShelfEntry</code> , <code>LookupKind</code>
3	the boundary	every place a value you did not write comes in, and what narrows it	cycles 1 and 3: <code>unknown</code> , then the guard
4	what must stay identical at runtime	three behaviours, and how you will check each	cycle 4: the three lines on the board
5	what I ask the tool, and what I check in its output	the request in two or three lines; four concrete things to look for	cycle 3: the three silencers

A spec written after the code only agrees with it. The proof is the git history: the commit of the spec comes before the commit of `ts/`.

ASSIGNMENT · 165–175

CLASS ASSIGNMENT 12: THE SAME MIGRATION, ON THE PANTRY

1 The spec, first · now, in class

`specs/ts-migration.md`, committed before `ts/` — no AI.

2 Types, guard, network · cycles 1 and 3

`value is Item` · `unknown at res.json()` · `readonly kind` · `'./types.js'`

3 The DOM · cycle 2

`$<T extends Element = Element>`, imported by one JavaScript file.

4 The review · cycle 3

two things the tool wrote and you changed, and why, in `PROMPTS.md`.

► **TEN MINUTES NOW:** part A, the whole spec, and its commit. The rest at home, **DUE BEFORE LESSON 13.** `ShelfEntry` is `Item`, `LookupError` is `ApiError` — nothing is copied; everything is recognised.



THE PROJECT IS DUE AT THE END OF THIS WEEK

BEFORE YOU PUSH

`node .checks/run-checks.mjs`
from `project/`. **The same checks**
CI runs, on your machine, in under
a minute.

THE CHECKLIST

`SUBMISSION_CHECKLIST-he.pdf`
— every requirement as something
you **do and observe**, not
something you believe.

THE ONE THAT COSTS THE MOST

Committing `ts/` and forgetting
`js/`. The browser loads the
output. **The grader does too.**

► Moodle: **REPO URL + COMMIT SHA**. The SHA freezes what is graded.

THREE QUESTIONS BEFORE YOU GO

- 1 You wrote `interface ShelfEntry` and compiled.
What is in the output file?
- 2 `tsc` printed zero errors and the page is blank.
Name two ways that happened today.
- 3 After cycle 1 you had migrated one file; the tsconfig includes `ts/**/*.*ts`.
By now, how many files does `tsc` check?



BEFORE NEXT WEEK

- 1 Submit the project.**
Repo URL + SHA, on Moodle, before the deadline. Run the self-check first.
- 2 Open your own project and read it once**
, end to end, as if somebody else wrote it. That is the exam.
- 3 Nothing to install.**
The lab machines will have your project preloaded and no internet. Bring nothing.

Next week you get three sentences you have now read twice. Only the application will be new — and it will be yours.

